TRABALHO DE ORDENAÇÃO

Disciplina SIN 213 - Projeto de Algoritmos

📌 **Data de Entrega:** 10 de novembro

📌 **Valor:** 20 vistos, distribuídos da seguinte forma:

- **10 vistos** – Relatório (entrega obrigatória via Moodle)
- **10 vistos** – Código (no Drive, organizado na sua pasta correspondente)

ENTREVISTA do merge e do quick

DIA 10 de novembro NO HORÁRIO DA AULA

(trabalho segue as mesmas orientações dos trabalhos anteriores)

Ordenação: Algoritmo1, Quick_Sort

Obs: o algoritmo 1 você deverá escolher 1 entre os algoritmos estudados: Insertion Sort, Selection Sort, Bubble sort, etc.

Objetivo

Algoritmo mais popular de ordenação.Na maioria dos casos ele roda em O(NlogN) para ordenações internas ou em memória. Para ordenar informações em arquivos em disco existem métodos melhores

FASES:

Implemente em linguagem C o algoritmo de ordenação
Algoritmo1
Quicksort_versao1(algoritmo nomal apresentado em sala de aula utilizando o primeiro elemento como pivô ou o do livro pagina 118, ou o do autor ziviani).
Quicksort_versao2(segundo algoritmo utilizando a media para melhorar a escolha do pivo).

Usando vetor e arquivo.

Implemente um Menu na aplicação para melhor iteração com o usuário. Esse Menu poder_a conter (sugestão):
As opções de IsertionSort - ex.(1: INSERTIONSORT EM VETOR, 2: INSERTIONSORT EM ARQUIVO).

A forma em que serão geradas as instâncias:

'r' para números randômicos.

'c' para números crescentes.

'd' para números randoms.

Opção de sair

Gere instâncias de tamanhos: 10, 100, 1.000, 10.000 e 100.000 e 1000000 para todas as formas (randômico, crescente e random), e execute o algoritmo implementado.
Na parte de arquivo, gere:
Um arquivo de entrada contendo as instâncias geradas.
Um arquivo contendo as instâncias ordenadas.
Um arquivo contendo o tempo gasto pela ordenação.

Os arquivos tanto os de entrada como os de saida devem ter na primeira linha o tamanho da instância e o restante dos dados, um por linha, como é indicado na Figura 1

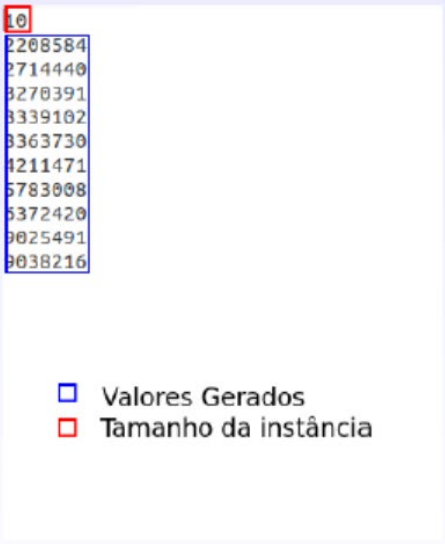


Figura: Exemplo de arquivo de entrada.

Os arquivos tanto os de entrada como os de tempo devem ter na primeira linha o tamanho da inst^ancia e o restante dos dados, um por linha, como _e indicado na Figura 1
Figura: Exemplo de arquivo de entrada.

Resultado Esperado

Criar um relatório contendo os resultados abaixo
(irão testar para as instancias : 10, 100, 1.000, 10.000 , 100.000, 1000000)
Que ira ser utilizada para fazer a tabela de comparação e os graficos)

Questões:

Primeiro resultado: Provar a análise de complexide do algoritmo 1 escolhido para o pior caso.

Segundo resultado: Responder porque o procedimento particiona do quick sort tem tempo linear O(n).

Terceiro resultado: calcular a complexidade do quicksort para os três casos, melhor, médio e pior caso

Quarto O quicksort tem ou não tem a fase de combinação, explique.

Quinto: Comparar todos algoritmos criando uma tabela de comparação.

Sexto: resultado:
Um gráfico para cada um dos algoritmos, quicksort versao 1,2. Cada um dos gráficos deve possuir duas curvas: uma representando o comportamento do algoritmo quando submetidos a dados em ordem descrescente e outra quando submetido a dados já ordenados.

Setimo resultado: Conclusão sobre os métodos de ordenação, utilizando os conceitos de complexidade

REFERÊNCIAS

Segue como organizar :

ORIENTAÇÕES PARA ELABORAÇÃO DOS CAPÍTULOS 65

O trabalho será desenvolvido **em capítulos**, sendo que o **Capítulo 6 será dedicado ao algoritmo Quick sort** Cada capítulo deverá conter obrigatoriamente as seguintes partes:

1. Introdução ao algoritmo

Apresente uma explicação sobre o algoritmo em questão, descrevendo seu funcionamento, sua finalidade e como o algoritmo realiza a ordenação dos elementos.

2. Teste de mesa manual

Realize um **teste de mesa manual**, passo a passo, utilizando uma entrada diferente dos exemplos trabalhados em sala de aula.

O teste deverá demonstrar claramente as alterações realizadas no vetor durante cada etapa da execução do algoritmo.

O teste de mesa deverá ser realizado manuscrito.

3. Análise e complexidade do algoritmo

Apresente a análise da complexidade do algoritmo em questão, considerando:

- melhor caso;
- caso médio;
- pior caso.

A análise deverá ser realizada conforme o procedimento apresentado em sala de aula para o algoritmo em questão, considerando o custo das instruções, o número de vezes que cada instrução é executada e os respectivos cálculos.

Esta parte deverá ser realizada manuscrita.

4. Tabelas e gráficos dos resultados

Realize os experimentos com os algoritmos e registre os tempos de execução obtidos para as diferentes instâncias.

As instâncias utilizadas deverão ser:

- 10;
- 100;
- 1.000;
- 10.000;
- 100.000;
- 1.000.000 elementos.

Os experimentos deverão considerar dados **já ordenados/Random/decrescente** .

Os resultados deverão ser apresentados por meio de tabelas e gráficos.

5. Gráficos individuais

Deverá ser elaborado **um gráfico para cada algoritmo**.

6. Tabela de comparação

Deverá ser elaborada uma **tabela de comparação dos algoritmos Insertion Sort, Selection Sort, Bubble Sort e Shellsort**.

A tabela deverá apresentar os tempos de execução obtidos para todas as instâncias utilizadas nos experimentos.

Os resultados deverão ser obtidos por meio da execução dos códigos dos algoritmos.

7. Gráfico geral

Deverá ser elaborado um **gráfico geral contendo os quatro algoritmos propostos**, permitindo a comparação de seus desempenhos para as diferentes instâncias.

O gráfico deverá considerar dados já ordenados e dados em ordem decrescente.

8. Conclusão

Deverá ser elaborada uma conclusão sobre os resultados obtidos nos experimentos.

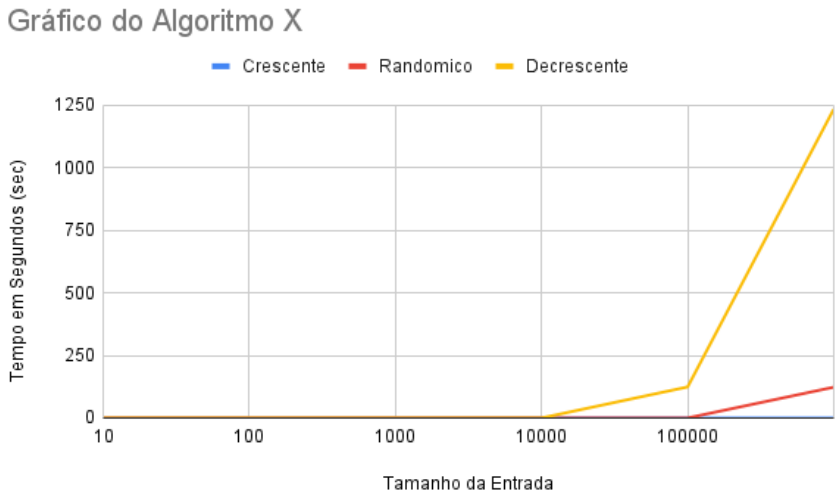


Figura 2: Exemplo de Gráfico do Algoritmo X com dados fictícios.

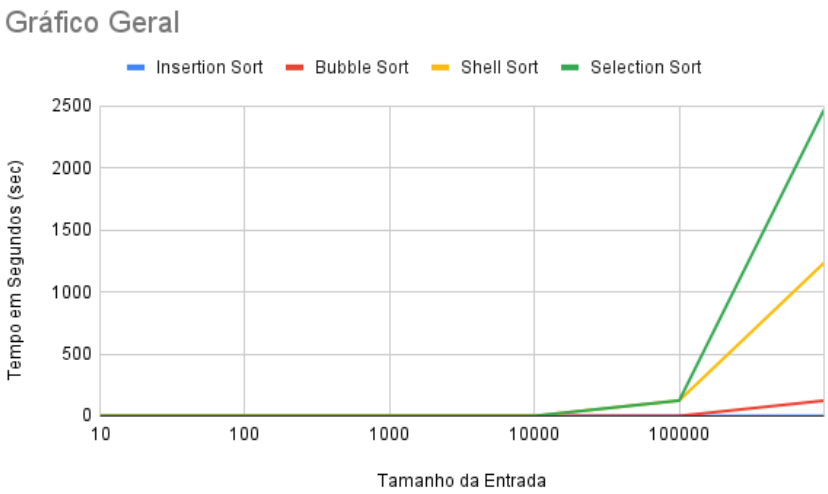


Figura 3: Exemplo de Gráfico Geral usando resultados de cada algoritmo usando dados fictícios.